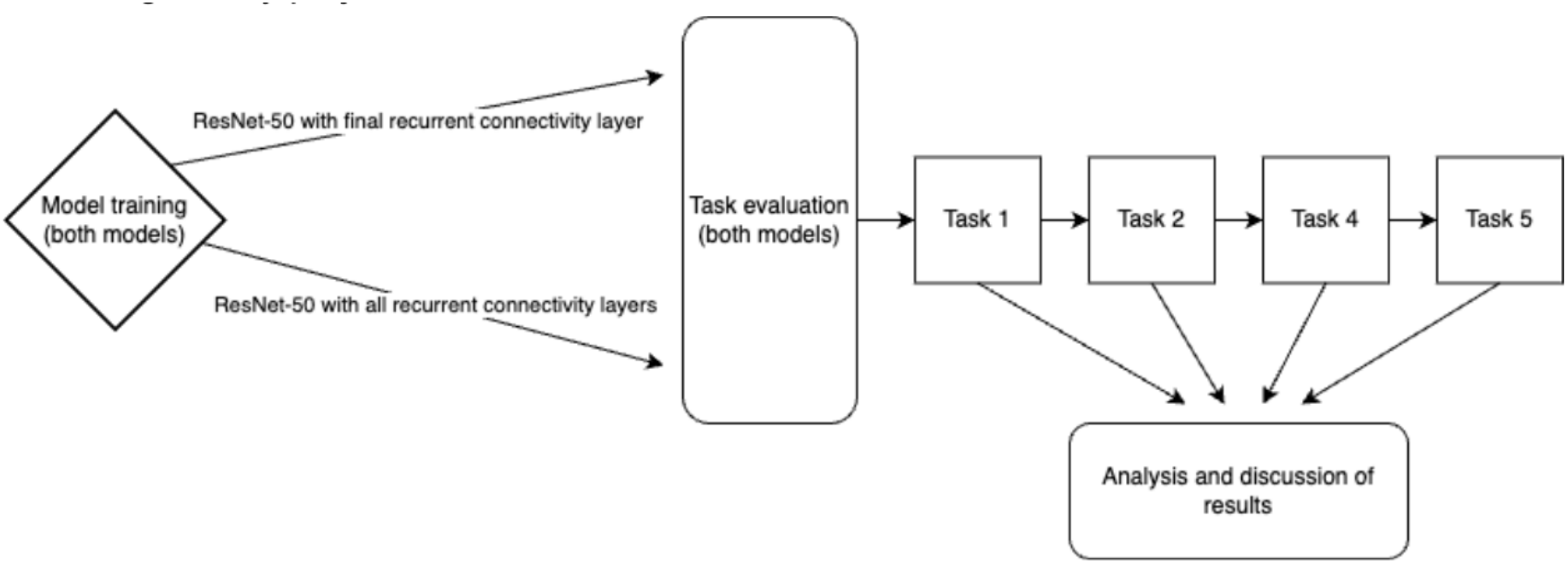
Close to the Finish Line — Working Memory

Neuro 240

April 8, 2025

Diego Luca Gonzalez Gauss

Recap


Model training
(both models)
ResNet-50 with final recurrent connectivity layer
ResNet-50 with all recurrent connectivity layers
Task evaluation
(both models)
Task 1
Task 2
Task 4
Task 5
Analysis and discussion of
results

Model Architectures

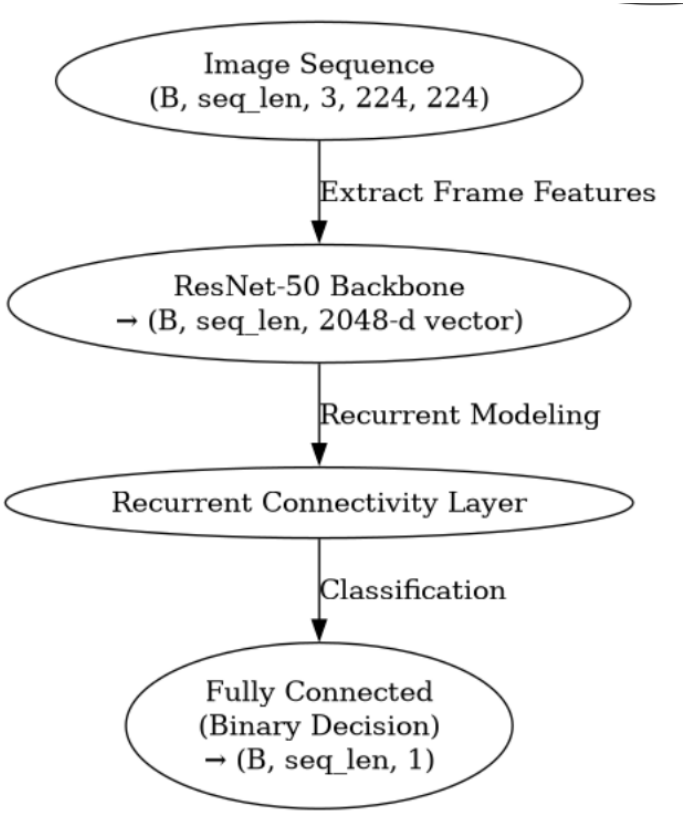


Figure: ResNet-50 with final connectivity layer

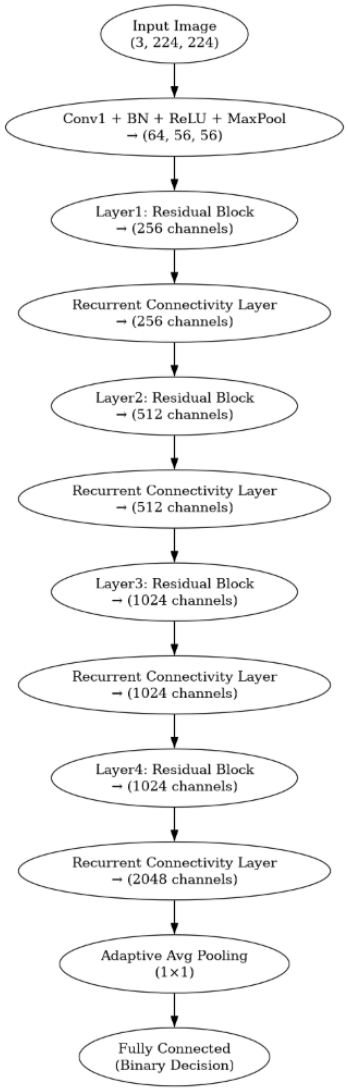


Figure: ResNet-50 with all connectivity layers

Methodology

```python
class DMTS_Dataset(Dataset):
    def __init__(self, base_dataset, task=1, delay_min=1, delay_max=5,
                 seq_min=5, seq_max=10, repetition_prob=0.3,
                 transform=None, test_transform=None):

        self.base_dataset = base_dataset
        self.task = task
        self.delay_min = delay_min
        self.delay_max = delay_max
        self.seq_min = seq_min
        self.seq_max = seq_max
        self.repetition_prob = repetition_prob
        self.transform = transform
        self.test_transform = test_transform
        self.indices = list(range(len(base_dataset)))

        avg_color = tuple(int(255 * c) for c in imagenet_mean)
        blank_pil = Image.new("RGB", (320, 320), avg_color)
        self.blank_frame = default_transform(blank_pil)

    def __len__(self):
        return 10000

    def get_random_image(self):
        idx = random.choice(self.indices)
        img, _ = self.base_dataset[idx]
        if self.transform and not isinstance(img, torch.Tensor):
            img = self.transform(img)
        return img

    def maybe_apply_test_transform(self, img_tensor):
        if self.test_transform is None:
            return img_tensor
        unnorm = transforms.Normalize(
            mean=[-m/s for m, s in zip(imagenet_mean, imagenet_std)],
            std=[1.0/s for s in imagenet_std]
        )(img_tensor.cpu())
        pil_img = transforms.ToPILImage()(unnorm)
        out = self.test_transform(pil_img)
        return out

    def __getitem__(self, idx):
        if self.task == 1:
            sample_img = self.get_random_image()
            delay_length = random.randint(self.delay_min, self.delay_max)
```

```python
            delay_imgs = [self.blank_frame for _ in range(delay_length)]
            is_repeat = random.random() < 0.5

            if is_repeat:
                test_img = sample_img.clone()
                if self.test_transform:
                    test_img = self.maybe_apply_test_transform(test_img)
            else:
                test_img = self.get_random_image()
                if self.test_transform:
                    test_img = self.maybe_apply_test_transform(test_img)
            sequence = [sample_img] + delay_imgs + [test_img]
            sequence = torch.stack(sequence, dim=0)
            label = torch.tensor([1.0]) if is_repeat else torch.tensor([0.0])
            return sequence, label

        elif self.task == 2:
            sample_img = self.get_random_image()
            delay_length = random.randint(self.delay_min, self.delay_max)
            delay_imgs = [self.get_random_image() for _ in range(delay_length)]
            is_repeat = random.random() < 0.5

            if is_repeat:
                test_img = sample_img.clone()
                if self.test_transform:
                    test_img = self.maybe_apply_test_transform(test_img)
            else:
                test_img = self.get_random_image()
                if self.test_transform:
                    test_img = self.maybe_apply_test_transform(test_img)
            sequence = [sample_img] + delay_imgs + [test_img]
            sequence = torch.stack(sequence, dim=0)
            label = torch.tensor([1.0]) if is_repeat else torch.tensor([0.0])
            return sequence, label

        elif self.task == 4:
            seq_length = random.randint(self.seq_min, self.seq_max)
```

```python
            seq_length = max(seq_length, 2)
            blank_img = self.get_random_image() * 0.0
            sequence = [blank_img for _ in range(seq_length)]
            sample_pos = random.randint(0, seq_length - 2)
            test_pos = random.randint(sample_pos + 1, seq_length - 1)
            sample_img = self.get_random_image()
            sequence[sample_pos] = sample_img
            is_repeat = random.random() < 0.5
            if is_repeat:
                test_img = sample_img.clone()
                if self.test_transform:
                    test_img = self.maybe_apply_test_transform(test_img)
            else:
                test_img = self.get_random_image()
                if self.test_transform:
                    test_img = self.maybe_apply_test_transform(test_img)
            sequence[test_pos] = test_img
            sequence = torch.stack(sequence, dim=0)
            label = torch.tensor([1.0]) if is_repeat else torch.tensor([0.0])
            return sequence, label

        elif self.task == 5:
            seq_length = random.randint(self.seq_min, self.seq_max)
            sequence = []
            labels = []
            for t in range(seq_length):
                if t == 0:
                    img = self.get_random_image()
                    label_t = 0.0
                else:
                    if random.random() < self.repetition_prob:
                        repeat_idx = random.randint(0, t - 1)
                        img = sequence[repeat_idx].clone()
                        label_t = 1.0
                    else:
                        img = self.get_random_image()
                        label_t = 0.0
                sequence.append(img)
                labels.append(torch.tensor([label_t]))
            sequence = torch.stack(sequence, dim=0)
            labels = torch.stack(labels, dim=0)
            return sequence, labels
```

```
Using device: cuda
```

Figure: Writing the dataset class and defining our tasks

Methodology (continued)

```python
# ONLY ONE LAYER
class RecurrentResNet(nn.Module):
    def __init__(self, hidden_size=256, num_layers=1, freeze_cnn=True, output_per_timestep=False):
        super(RecurrentResNet, self).__init__()
        from torchvision.models import resnet50, ResNet50_Weights
        resnet = resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
        modules = list(resnet.children())[:-1]
        self.cnn = nn.Sequential(*modules)
        if freeze_cnn:
            for param in self.cnn.parameters():
                param.requires_grad = False

        self.lstm = nn.LSTM(input_size=2048, hidden_size=hidden_size, num_layers=num_layers, batch_first=True)
        self.output_per_timestep = output_per_timestep
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        batch_size, seq_len, C, H, W = x.size()
        features = []
        for t in range(seq_len):
            img = x[:, t, :, :, :]
            feat = self.cnn(img)
            feat = feat.view(batch_size, -1)
            features.append(feat.unsqueeze(1))
        features = torch.cat(features, dim=1)
        lstm_out, _ = self.lstm(features)
        if self.output_per_timestep:
            out = self.fc(lstm_out)
        else:
            out = self.fc(lstm_out[:, -1, :])
        return out
```

Figure: Code for the final connectivity layer model

```python
class ConvLSTMCell(nn.Module):
    def __init__(self, input_dim, hidden_dim, kernel_size, bias=True):
        super(ConvLSTMCell, self).__init__()
        padding = (kernel_size[0] // 2, kernel_size[1] // 2)
        self.conv = nn.Conv2d(input_dim + hidden_dim, 4 * hidden_dim,
                              kernel_size, padding=padding, bias=bias)
        self.hidden_dim = hidden_dim

    def forward(self, x, state):
        h, c = state
        combined = torch.cat([x, h], dim=1)
        conv_out = self.conv(combined)
        cc_i, cc_f, cc_o, cc_g = torch.split(conv_out, self.hidden_dim, dim=1)
        i = torch.sigmoid(cc_i)
        f = torch.sigmoid(cc_f)
        o = torch.sigmoid(cc_o)
        g = torch.tanh(cc_g)
        c_next = f * c + i * g
        h_next = o * torch.tanh(c_next)
        return h_next, c_next

    def init_hidden(self, batch_size, spatial_size):
        height, width = spatial_size
        h = torch.zeros(batch_size, self.hidden_dim, height, width, device=self.conv.weight.device)
        c = torch.zeros(batch_size, self.hidden_dim, height, width, device=self.conv.weight.device)
        return (h, c)

class InterleavedRecurrentBlock(nn.Module):
    def __init__(self, channels, recurrence_steps=2, kernel_size=(3,3)):
        super(InterleavedRecurrentBlock, self).__init__()
        self.recurrence_steps = recurrence_steps
        self.cell = ConvLSTMCell(input_dim=channels, hidden_dim=channels, kernel_size=kernel_size)

    def forward(self, x):
        # x: (B, channels, H, W)
        B, C, H, W = x.size()
        h, c = self.cell.init_hidden(B, (H, W))
        out = x
        for _ in range(self.recurrence_steps):
            h, c = self.cell(out, (h, c))
            out = h
        return out

class ResNet50_InterleavedRecurrent(nn.Module):
    def __init__(self, recurrence_steps=2):
        super(ResNet50_InterleavedRecurrent, self).__init__()
        weights = ResNet50_Weights.IMAGENET1K_V1
        resnet = resnet50(weights=weights)
        self.conv1 = resnet.conv1
        self.bn1 = resnet.bn1
        self.relu = resnet.relu
        self.maxpool = resnet.maxpool
        self.layer1 = resnet.layer1  # output: (B, 256, 56, 56)
        self.layer2 = resnet.layer2  # output: (B, 512, 28, 28)
        self.layer3 = resnet.layer3  # output: (B, 1024, 14, 14)
        self.layer4 = resnet.layer4  # output: (B, 2048, 7, 7)
        self.interleaved1 = InterleavedRecurrentBlock(channels=256, recurrence_steps=recurrence_steps)
        self.interleaved2 = InterleavedRecurrentBlock(channels=512, recurrence_steps=recurrence_steps)
        self.interleaved3 = InterleavedRecurrentBlock(channels=1024, recurrence_steps=recurrence_steps)
        self.interleaved4 = InterleavedRecurrentBlock(channels=2048, recurrence_steps=recurrence_steps)
        self.avgpool = resnet.avgpool
        num_ftrs = resnet.fc.in_features
        self.fc = nn.Linear(num_ftrs, 1)

    def forward(self, x):
        # x: (B, 3, 224, 224)
        x = self.conv1(x)       # (B, 64, 112, 112)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)     # (B, 64, 56, 56)

        x = self.layer1(x)      # (B, 256, 56, 56)
        x = self.interleaved1(x)  # (B, 256, 56, 56)
        x = self.layer2(x)      # (B, 512, 28, 28)
        x = self.interleaved2(x)  # (B, 512, 28, 28)
        x = self.layer3(x)      # (B, 1024, 14, 14)
        x = self.interleaved3(x)  # (B, 1024, 14, 14)
        x = self.layer4(x)      # (B, 2048, 7, 7)
        x = self.interleaved4(x)  # (B, 2048, 7, 7)
        x = self.avgpool(x)     # (B, 2048, 1, 1)
        x = torch.flatten(x, 1)
        x = self.fc(x)          # (B, 1)
        return x
```

Figure: Code for the all connectivity layers model

Generalizing to Task 1 and Task 2

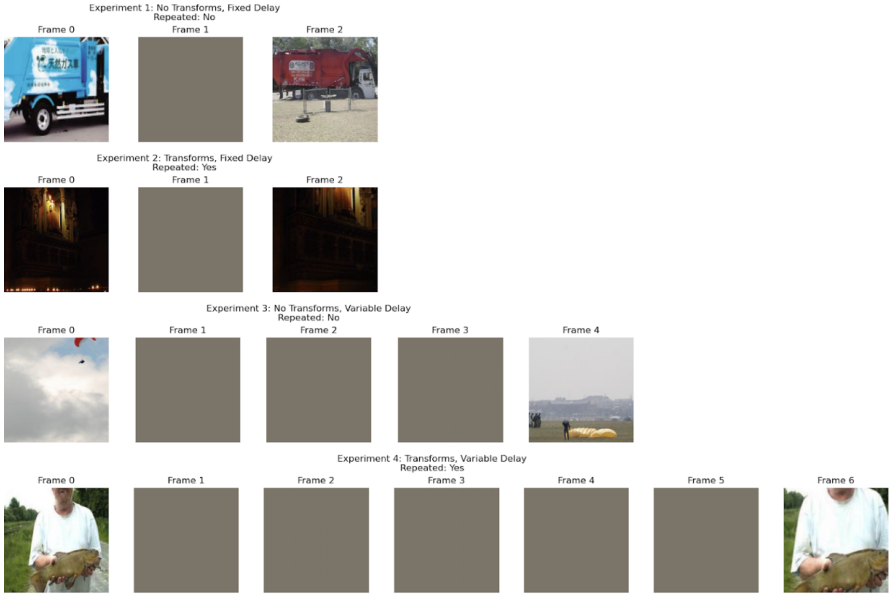


Figure: Experiments to task 1

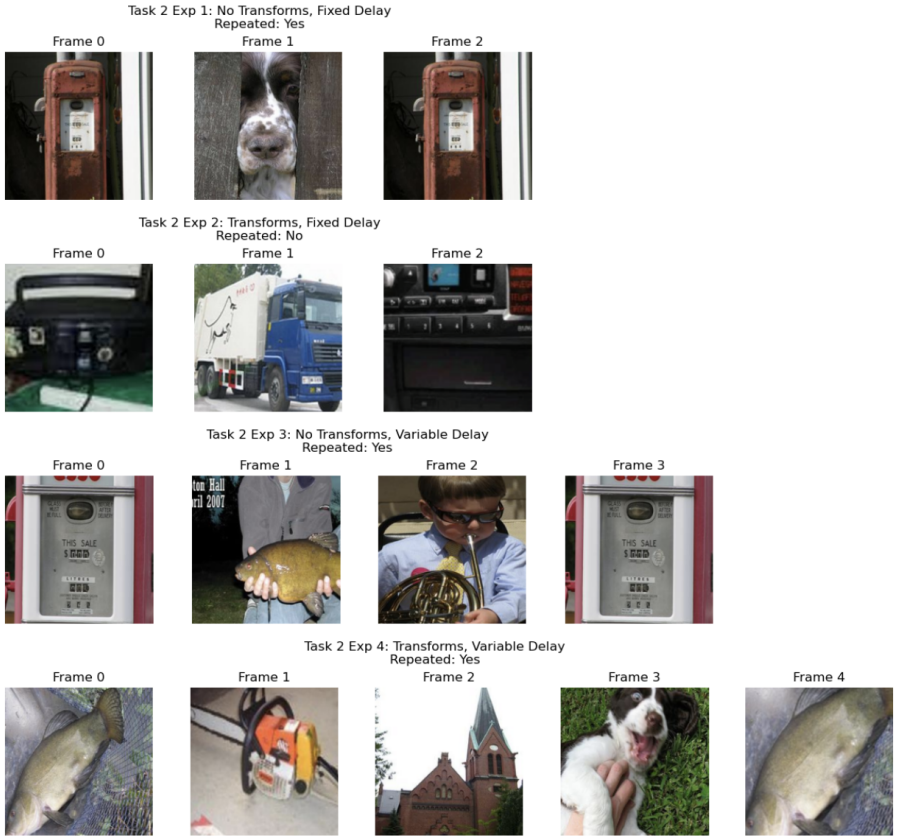


Figure: Experiments to task 2

Task 1 and 2 Results — ResNet-50 with Recurrent Connectivity on All Layers

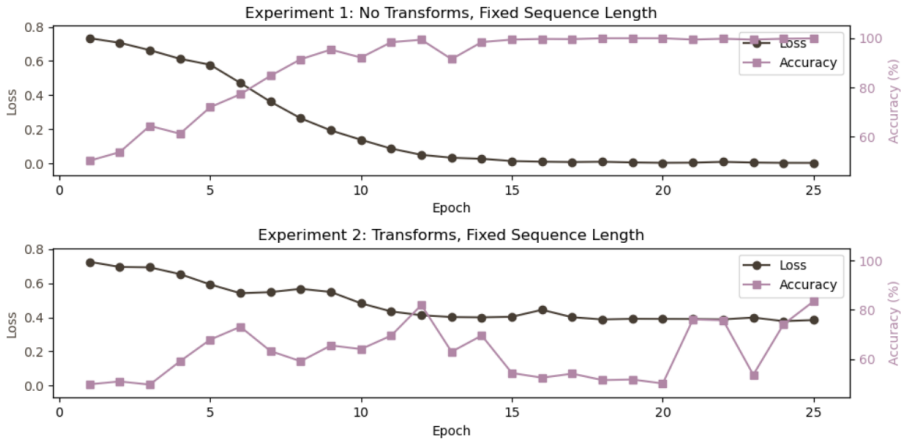


Figure: Accuracies and losses for task 1 experiments 1 and 2

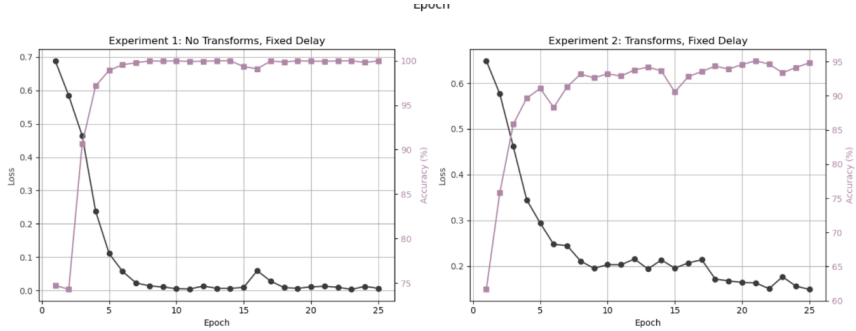


Figure: Accuracies and losses for task 2 experiments 1 and 2

Task 1 and 2 Results — ResNet-50 with Recurrent Connectivity on All Layers (cont.)

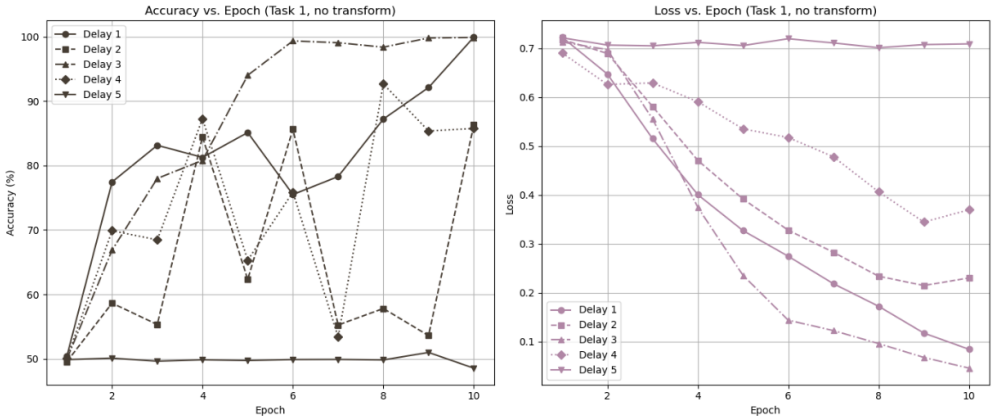


Figure: Accuracies and losses for task 1 experiment 3

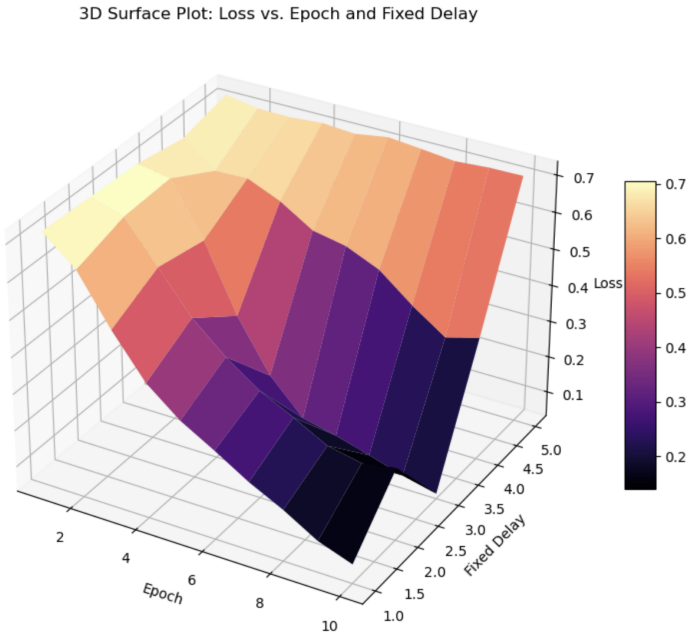


Figure: Surface plot for loss, task 1

Task 1 and 2 Results — ResNet-50 with Recurrent Connectivity Before Output

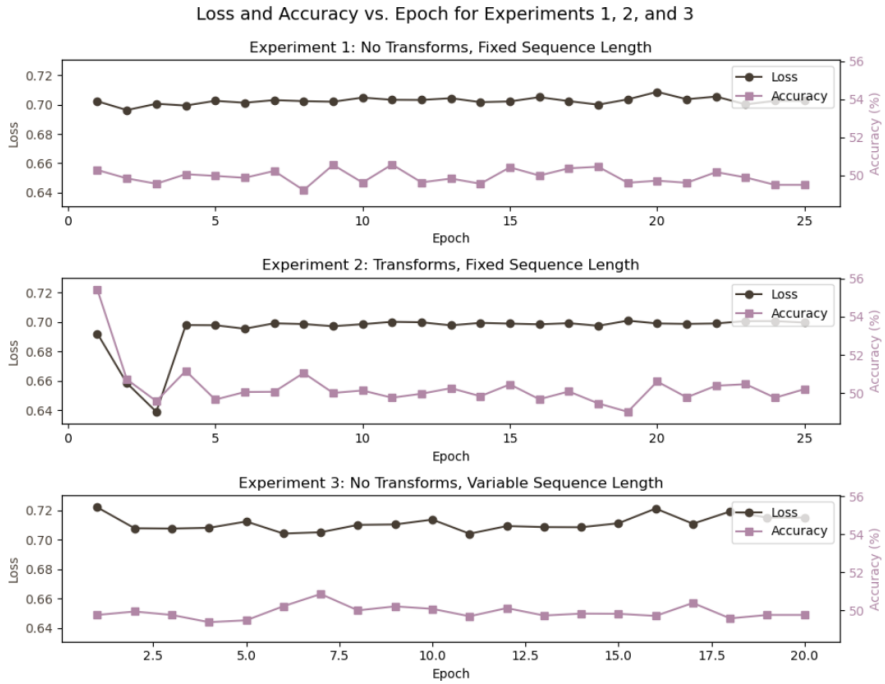


Figure: Accuracies and losses for task 1 experiments 1-3

What's next

- Running code and visualization
  - Run fully generalized task 1 and 2
  - Run tasks 4 and 5
- Further hyperparameter tuning
- Prospective alternate models?

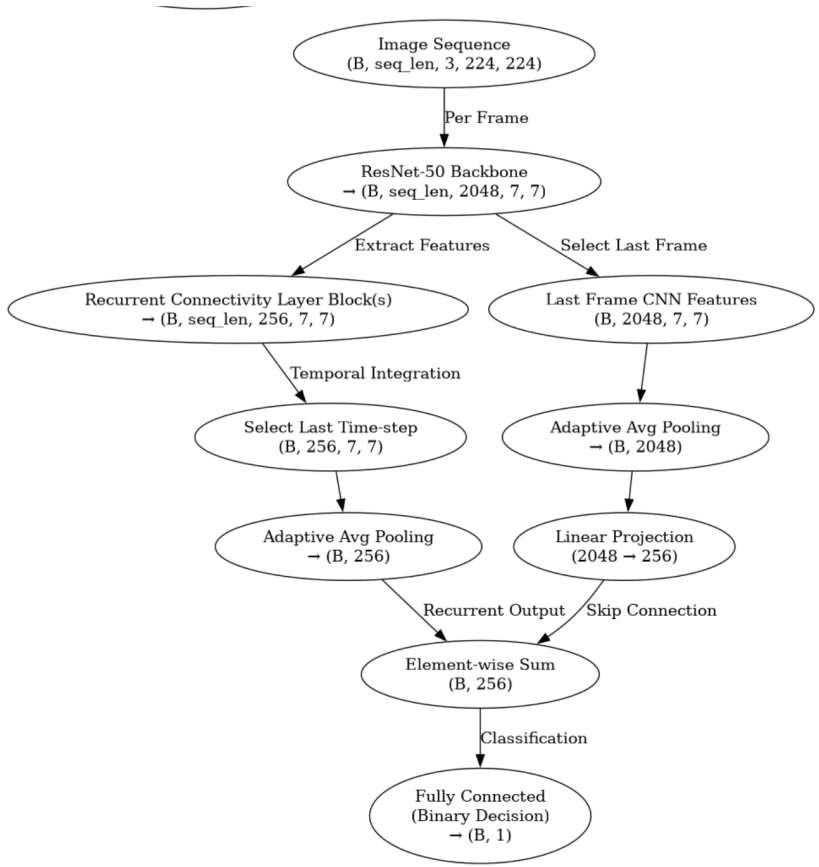


Figure: Proposed alternate